

# Distributed Systems

## Unit III: Inter-Process Communication

### Syllabus:

**IPC:** Introduction, Layered protocols, API for internet protocols, IPC through shared memory, external data representation and marshaling, Types of communication, inter process communication, multicast communication, message-oriented communication, MPI, network virtualization, overlay networks

**Coordination:** Clock synchronization, logical clocks, mutual exclusion, election algorithms, Gossip based coordination

**Case Study:** IBM WebSphere Message Queuing

#### ♦ IPC

##### ➤ Introduction

#### Introduction to Inter-Process Communication (IPC)

**Inter-Process Communication (IPC)** is a mechanism that allows different processes running in an operating system to communicate and share data with each other. Since processes run independently and have separate memory spaces, IPC provides methods for exchanging information and synchronizing actions between them.

IPC is essential in modern operating systems because many applications require multiple processes to work together. For example, a web browser may use separate processes for tabs, extensions, and rendering, all of which need to communicate efficiently.

#### Why IPC is Needed

- Data sharing between processes
- Process synchronization
- Resource sharing
- Improving modularity and performance
- Communication between applications and services

#### Types of IPC Mechanisms

##### 1. Pipes

- a. Used for communication between related processes
- b. Can be unidirectional or bidirectional

## **2. Message Queues**

- a. Processes exchange messages through queues managed by the OS

## **3. Shared Memory**

- a. Multiple processes access the same memory region
- b. Fastest IPC method

## **4. Sockets**

- a. Used for communication between processes on the same or different systems over a network

## **5. Semaphores**

- a. Used for synchronization and controlling access to shared resources

## **6. Signals**

- a. Simple notifications sent to processes to indicate events

### Advantages of IPC

- Enables cooperation between processes
- Improves system efficiency
- Supports multitasking and parallel processing
- Allows modular application design

### Disadvantages of IPC

- Can increase system complexity
- Synchronization issues may occur
- Security and data consistency must be managed carefully

### Applications of IPC

- Client-server applications
- Operating systems
- Distributed systems
- Multi-threaded and multi-process applications
- Real-time systems

In summary, IPC is a fundamental concept in operating systems that enables processes to communicate, coordinate, and share resources effectively, ensuring smooth and efficient system operation.

### ➤ **Layered Protocols**

## Layered Protocols in Inter-Process Communication (IPC)

Layered protocols in IPC refer to the organization of communication functions into multiple layers, where each layer performs a specific task and interacts only with adjacent layers.

This layered approach simplifies communication design, improves modularity, and makes systems easier to maintain and develop.

The concept is similar to networking models such as the OSI Model.

### Purpose of Layered Protocols

- Divide complex communication tasks into manageable parts
- Provide standard communication rules
- Ensure reliable and organized data exchange
- Improve flexibility and scalability

### Common Layers in IPC Communication

#### 1. Application Layer

- Interface between user applications and IPC services
- Handles user requests and data generation

**Example:** Chat applications, file-sharing programs

#### 2. Presentation Layer

- Converts data into a suitable format
- Handles encryption, compression, and translation

##### **Functions:**

- Data formatting
- Encryption/decryption
- Compression

#### 3. Session Layer

- Establishes, maintains, and terminates communication sessions between processes

##### **Functions:**

- Session synchronization
- Connection management

#### 4. Transport Layer

- Ensures reliable data transfer between processes
- Handles error detection and flow control

##### **Functions:**

- Data segmentation
- Error recovery
- Process-to-process delivery

#### 5. Network/Communication Layer

- Manages routing and communication paths
- Supports communication between systems if processes are on different machines

#### 6. Physical/Data Transfer Layer

- Handles actual transmission of data through hardware or system resources

#### How Layered Protocols Work in IPC

When one process sends data:

1. Data moves from the application layer downward through each layer
2. Each layer adds its own control information
3. Data is transmitted to another process
4. The receiving process removes layer information in reverse order

This process is called:

- **Encapsulation** (sender side)
- **Decapsulation** (receiver side)

#### Advantages of Layered Protocols

- Easier debugging and maintenance
- Better modularity
- Standardization

- Independent layer development
- Improved scalability

### Disadvantages

- Additional overhead
- Increased processing time
- Complex implementation in some systems

### Example in IPC

In a client-server system:

- The application layer handles user requests
- The transport layer ensures reliable message delivery
- Lower layers manage data transfer between processes or systems

Thus, layered protocols provide a structured and efficient way for processes to communicate reliably in IPC systems.

### ➤ API for internet protocols

#### API for Internet Protocols in IPC

An **Application Programming Interface (API)** for Internet Protocols in IPC provides a set of functions and rules that allow processes to communicate over a network using standard internet protocols such as TCP/IP and UDP.

These APIs help applications send and receive data between processes running on the same machine or different systems connected through a network.

### Socket API

The most commonly used API for internet-based IPC is the **Socket API**.

A **socket** is an endpoint used for communication between two processes over a network.

It supports protocols such as:

- **TCP (Transmission Control Protocol)** — reliable communication
- **UDP (User Datagram Protocol)** — fast but unreliable communication

## Components of Socket API

### 1. Socket Creation

Creates a communication endpoint.

`socket()`

#### Parameters

- Domain (IPv4/IPv6)
- Type (TCP/UDP)
- Protocol

### 2. Binding

Associates a socket with an IP address and port number.

`bind()`

### 3. Listening

Used by servers to wait for client connections.

`listen()`

### 4. Accepting Connections

Accepts incoming client requests.

`accept()`

### 5. Connecting

Used by clients to establish connection with a server.

`connect()`

## 6. Sending Data

Transfers data between processes.

`send()`

`write()`

## 7. Receiving Data

Receives transmitted data.

`recv()`

`read()`

## 8. Closing Socket

Terminates communication.

`close()`

## Communication Models

### TCP/IP Model

Provides reliable, connection-oriented communication.

Features:

- Error checking
- Ordered delivery
- Reliable transmission

Used in:

- Web browsing

- Email
- File transfer

## UDP Model

Provides connectionless communication.

Features:

- Faster transmission
- No guarantee of delivery
- Low overhead

Used in:

- Video streaming
- Online gaming
- Voice calls

## Example of Socket Communication

### Server Process

1. Create socket
2. Bind socket
3. Listen for connections
4. Accept client request
5. Exchange data

### Client Process

1. Create socket
2. Connect to server
3. Send/receive data

## Advantages of Internet Protocol APIs

- Enables communication across networks
- Supports distributed systems
- Standardized communication mechanism
- Efficient client-server interaction

## Disadvantages

- Complex programming
- Security concerns
- Network delays may affect performance

## Applications

- Client-server applications
- Web services
- Distributed computing
- Cloud applications
- Remote login systems

In summary, APIs for internet protocols in IPC mainly use socket programming to enable communication between processes over networks using protocols like TCP and UDP.

### ➤ **IPC through shared memory**

## IPC Through Shared Memory

**Shared Memory** is one of the fastest methods of **Inter-Process Communication (IPC)**. In this method, two or more processes share a common region of memory to exchange data directly.

Instead of sending messages through the operating system repeatedly, processes can read and write to the same memory area, making communication very efficient.

## Working of Shared Memory

1. The operating system creates a shared memory segment.
2. Multiple processes attach to this memory segment.
3. Processes read from and write to the shared memory.
4. After communication is complete, the memory segment is detached and removed.

## Basic Concept

Process A <---- Shared Memory ----> Process B

Both processes access the same physical memory region.

## Steps in Shared Memory IPC

### 1. Create Shared Memory Segment

A process creates a shared memory block using system calls such as:

```
shmget()
```

### 2. Attach Shared Memory

Processes attach the shared memory to their address space.

```
shmat()
```

### 3. Read/Write Data

Processes directly access the memory using pointers.

Example:

```
strcpy(shared_memory, "Hello");
```

### 4. Detach Shared Memory

After use, processes detach the memory.

```
shmdt()
```

## 5. Remove Shared Memory

The shared segment is deleted from the system.

`shmctl()`

### Advantages of Shared Memory

- Very fast communication
- Efficient for large data transfer
- Minimal CPU overhead
- Direct memory access

### Disadvantages

- Synchronization problems may occur
- Risk of data corruption
- More complex programming
- Requires proper memory management

### Synchronization in Shared Memory

Since multiple processes may access the same memory simultaneously, synchronization mechanisms are needed:

- **Semaphores**
- **Mutex locks**
- **Monitors**

These prevent race conditions and ensure data consistency.

### Applications of Shared Memory

- Producer-consumer problems
- Multimedia systems
- Real-time systems
- High-performance computing

- Database management systems

### Example Use Case

A producer process writes data into shared memory, and a consumer process reads it without copying data through the kernel repeatedly.

Producer → Shared Memory → Consumer

### Comparison with Other IPC Methods

| IPC Method     | Speed     | Complexity |
|----------------|-----------|------------|
| Shared Memory  | Very High | Moderate   |
| Pipes          | Medium    | Low        |
| Message Queues | Medium    | Moderate   |
| Sockets        | Lower     | Higher     |

In summary, shared memory is a high-speed IPC mechanism where processes communicate by accessing a common memory area. It is widely used when fast and efficient data sharing is required between processes.

### ➤ External Data Representation and Marshaling

May-Jun 2024

Q1) a) Explain external data representation (XDR), marshalling and unmarshalling. Why is XDR required? Discuss in brief the three alternative approaches to external data representation.

May-Jun 2025

Q2) a) Define External Data Representation, Marshalling and Unmarshalling. Discuss the three alternative approaches to XDR and marshalling.

Ans. External Data Representation (XDR)

**External Data Representation (XDR)** is a standard format used to represent data in a machine-independent manner during communication between different computer systems.

Different systems may store data differently because of:

- Different processor architectures
- Different byte ordering (big-endian/little-endian)
- Different character encodings
- Different data type sizes

XDR ensures that data exchanged between processes or systems is interpreted correctly regardless of hardware or operating system differences.

It is commonly used in distributed systems and Remote Procedure Calls (RPC).

### Why is XDR Required?

Different computer systems represent data internally in different ways.

For example:

- Intel systems use **little-endian**
- Some network systems use **big-endian**

If data is sent directly without conversion, the receiving system may misinterpret it.

XDR solves this problem by:

- Providing a common standard format
- Ensuring portability
- Enabling interoperability between heterogeneous systems

### Marshalling and Unmarshalling

#### Marshalling

**Marshalling** is the process of converting data and objects from internal memory format into a standard transmittable format (such as XDR) before sending it over a network.

### Steps:

1. Collect data
2. Convert data into external format
3. Pack data into a message

### Example

Internal Data → XDR Format → Transmission

### Unmarshalling

**Unmarshalling** is the reverse process.

It converts received external-format data back into the receiver's internal machine format.

### Steps:

1. Receive message
2. Extract packed data
3. Convert XDR data into local representation

### Example

Received XDR Data → Local Machine Format

### Working of XDR

Sender System



Marshalling



XDR Encoding



Network Transmission



XDR Decoding



Unmarshalling



## Receiver System

### Advantages of XDR

- Machine independence
- Portability across platforms
- Reliable communication
- Supports heterogeneous distributed systems

### Alternative Approaches to External Data Representation

There are three main approaches:

#### 1. Sender Makes Conversion

In this approach:

- The sender converts data into the receiver's machine format before transmission.

#### Working

Sender Format → Receiver Format → Send

#### Advantages

- Receiver performs less work

#### Disadvantages

- Sender must know receiver architecture
- Difficult in large distributed systems
- Poor scalability

#### 2. Receiver Makes Conversion

In this approach:

- Sender transmits data in its own internal format.

- Receiver converts data into its local format.

#### Working

Sender Native Format → Transmission → Receiver Converts

#### Advantages

- Simple sender implementation

#### Disadvantages

- Receiver must understand all sender formats
- High complexity for heterogeneous systems

### 3. Use Standard External Representation (XDR Approach)

In this method:

- Both sender and receiver convert data to and from a common standard format.

#### Working

Sender Format → XDR → Receiver Format

#### Advantages

- Highly portable
- Scalable
- Easy interoperability
- Widely used in distributed systems

#### Disadvantages

- Extra conversion overhead

## Comparison of the Three Approaches

| Approach            | Conversion Done By     | Scalability | Complexity |
|---------------------|------------------------|-------------|------------|
| Sender Conversion   | Sender                 | Low         | High       |
| Receiver Conversion | Receiver               | Low         | High       |
| XDR Standard Format | Both Sender & Receiver | High        | Moderate   |

## Applications of XDR

- Remote Procedure Calls (RPC)
- Distributed systems
- Network communication
- Client-server applications

In summary, XDR is a machine-independent data representation standard used in distributed systems. Marshalling converts local data into XDR format, while unmarshalling converts it back into machine-specific format. XDR is required to ensure reliable communication between heterogeneous systems.

### ➤ Types of Communication

## Types of Communication in Inter-Process Communication (IPC)

In IPC, communication between processes can occur in different ways depending on how data is transferred and synchronized. The main types of communication are as follows:

### 1. Direct Communication

In direct communication, processes communicate directly with each other by explicitly naming the sender or receiver.

#### Features

- Sender and receiver must know each other
- Communication link is established automatically
- Usually one-to-one communication

### Example

Process A → Process B

### Advantages

- Simple implementation
- Faster communication

### Disadvantages

- Tight coupling between processes
- Less flexible

## 2. Indirect Communication

In indirect communication, processes communicate through an intermediary such as a mailbox or message queue.

### Features

- Processes do not need to know each other directly
- Communication occurs through shared mailboxes

### Example

Process A → Mailbox → Process B

### Advantages

- Flexible communication
- Supports one-to-many communication

### Disadvantages

- More complex
- Additional overhead

## 3. Synchronous Communication

Also called **blocking communication**.

The sender and receiver wait until the message transfer is completed.

### Features

- Processes synchronize during communication
- Sender may block until receiver receives message

### Advantages

- Reliable communication
- Easier synchronization

### Disadvantages

- Processes may remain idle while waiting

## 4. Asynchronous Communication

Also called **non-blocking communication**.

The sender continues execution after sending the message without waiting for the receiver.

### Features

- Independent execution
- Messages stored temporarily in buffers or queues

### Advantages

- Better performance
- Higher concurrency

### Disadvantages

- More difficult synchronization
- Possible message overflow

## 5. Message Passing Communication

Processes exchange messages using system calls.

### Characteristics

- No shared memory required
- Suitable for distributed systems

### Example IPC Mechanisms

- Pipes

- Message queues
- Sockets

## 6. Shared Memory Communication

Processes communicate by accessing a common shared memory area.

### Characteristics

- Fastest IPC method
- Requires synchronization mechanisms

### Example

Process A ↔ Shared Memory ↔ Process B

## 7. Half-Duplex Communication

Communication occurs in both directions, but only one direction at a time.

### Example

- Walkie-talkies

## 8. Full-Duplex Communication

Communication occurs simultaneously in both directions.

### Example

- Telephone conversation

## Comparison Table

| Type            | Communication Style   | Speed     | Complexity |
|-----------------|-----------------------|-----------|------------|
| Direct          | Process-to-process    | Fast      | Low        |
| Indirect        | Through mailbox       | Moderate  | Moderate   |
| Synchronous     | Blocking              | Moderate  | Low        |
| Asynchronous    | Non-blocking          | Fast      | High       |
| Shared Memory   | Shared data area      | Very Fast | Moderate   |
| Message Passing | Send/receive messages | Moderate  | Moderate   |

## Applications

- Operating systems
- Distributed systems
- Client-server applications
- Real-time systems
- Parallel computing

In summary, IPC supports multiple types of communication such as direct, indirect, synchronous, asynchronous, shared memory, and message passing, each suited for different system requirements and performance needs.

### ➤ Inter Process Communication

**May-Jun 2023**

Q1) a) What are the general characteristics of inter-process communication? What are various types of Synchronous and asynchronous communication in IPC? Why it is blocking receive has no disadvantages in Java.

Ans. General Characteristics of Inter-Process Communication (IPC)

Inter-Process Communication (IPC) enables processes to exchange data and coordinate their activities. The general characteristics of IPC are:

## 1. Communication Between Processes

IPC allows independent processes to share information and interact with each other.

## 2. Data Exchange

Processes can transfer:

- Messages
- Signals
- Shared data
- Files

## 3. Synchronization

IPC provides synchronization mechanisms to coordinate process execution and avoid conflicts.

Examples:

- Semaphores
- Mutexes
- Monitors

## 4. Process Coordination

Processes can cooperate to perform complex tasks efficiently.

## 5. Resource Sharing

IPC allows multiple processes to share:

- Memory
- Files
- Devices
- Network resources

## 6. Support for Distributed Systems

IPC enables communication between processes running on different computers connected through networks.

## 7. Protection and Security

Operating systems control permissions and access to shared resources to prevent unauthorized access.

## 8. Efficiency

Efficient IPC mechanisms reduce communication overhead and improve system performance.

## Synchronous and Asynchronous Communication in IPC

IPC communication can mainly be classified into:

### 1. Synchronous Communication (Blocking Communication)

In synchronous communication, the sender or receiver waits until communication is completed.

#### Types of Synchronous Communication

##### a) Blocking Send

- Sender waits until the message is received or copied.

Sender waits → Receiver gets message

##### b) Blocking Receive

- Receiver waits until a message arrives.

Receiver waits ← Message arrives

#### Advantages

- Easy synchronization
- Reliable communication
- Simpler implementation

## Disadvantages

- Processes may remain idle while waiting
- Reduced concurrency

## 2. Asynchronous Communication (Non-Blocking Communication)

In asynchronous communication, processes continue execution without waiting.

### Types of Asynchronous Communication

#### a) Non-Blocking Send

- Sender sends message and immediately continues execution.

#### b) Non-Blocking Receive

- Receiver checks for messages and continues if none are available.

Sender → Continue execution

Receiver → Continue if no message

## Advantages

- Better parallelism
- Higher system performance
- Improved responsiveness

## Disadvantages

- More complex programming
- Requires buffer management and synchronization

## Comparison Between Synchronous and Asynchronous Communication

| Feature         | Synchronous | Asynchronous |
|-----------------|-------------|--------------|
| Waiting         | Required    | Not required |
| Execution       | Blocking    | Non-blocking |
| Complexity      | Simpler     | More complex |
| Performance     | Lower       | Higher       |
| Synchronization | Easier      | Difficult    |

### Why Blocking Receive Has No Major Disadvantages in Java

In many systems, blocking receive can cause CPU wastage because a process waits idly for data. However, in Java, blocking receive generally has no significant disadvantages because of Java's efficient threading and runtime management.

#### Reasons

##### 1. Multithreading Support

Java supports lightweight threads.

While one thread waits for data, other threads continue execution.

##### 2. Efficient Thread Scheduling

The Java Virtual Machine (JVM) and operating system efficiently schedule blocked threads without wasting CPU resources.

##### 3. No Busy Waiting

Java blocking operations suspend the waiting thread instead of continuously checking for data.

Thus, CPU time is not wasted.

#### 4. Simpler Programming

Blocking receive simplifies communication logic because programmers do not need complex polling mechanisms.

Example:

```
socket.receive(packet);
```

The thread sleeps until data arrives.

#### 5. Better Resource Utilization

Java's thread management allows other tasks to execute while one thread is blocked.

#### Conclusion

IPC enables communication and synchronization between processes through various mechanisms. Communication can be synchronous (blocking) or asynchronous (non-blocking). In Java, blocking receive does not create major disadvantages because Java efficiently handles blocked threads using JVM scheduling and multithreading support.

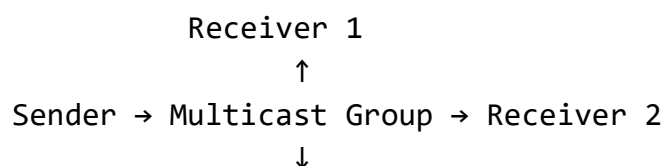
#### ➤ Multicast Communication

##### Multicast Communication

**Multicast communication** is a communication method in which a single sender transmits data to a selected group of receivers simultaneously.

Instead of sending separate copies of data to every receiver, multicast sends one copy that is distributed to multiple recipients efficiently through the network.

##### Basic Concept



## Receiver 3

The sender sends data to a **multicast group address**, and all members of the group receive the message.

### Characteristics of Multicast Communication

- One-to-many communication
- Efficient bandwidth usage
- Group-based communication
- Dynamic group membership
- Simultaneous data delivery

### How Multicast Works

1. A multicast group is created.
2. Receivers join the group.
3. Sender transmits data to the multicast address.
4. Network routers distribute packets to all group members.

### Multicast Addressing

In IP networking:

- Special IP addresses are reserved for multicast communication.

For IPv4:

224.0.0.0 to 239.255.255.255

These addresses identify multicast groups.

## Types of Communication

| Type      | Description                 |
|-----------|-----------------------------|
| Unicast   | One sender → One receiver   |
| Broadcast | One sender → All devices    |
| Multicast | One sender → Selected group |

## Advantages of Multicast Communication

### 1. Efficient Bandwidth Usage

Only one copy of data is transmitted initially.

### 2. Reduced Network Traffic

Avoids repeated transmissions to multiple receivers.

### 3. Scalability

Supports communication with many receivers efficiently.

### 4. Simultaneous Delivery

All group members receive data nearly at the same time.

## Disadvantages

- Complex implementation
- Requires multicast-enabled routers
- Security challenges
- Difficult error handling

## Applications of Multicast Communication

- Video conferencing
- Live video streaming
- Online gaming
- Stock market updates
- Distributed databases
- Real-time data distribution

## Multicast in IPC

In IPC and distributed systems, multicast communication allows one process to communicate with multiple processes simultaneously.

Example:

Server → Multiple Client Processes

This improves efficiency in distributed applications.

## Multicast vs Broadcast

| Feature         | Multicast      | Broadcast    |
|-----------------|----------------|--------------|
| Receivers       | Selected group | All devices  |
| Efficiency      | High           | Lower        |
| Bandwidth Usage | Optimized      | More traffic |
| Scalability     | Better         | Limited      |

## Example in Java

Java supports multicast using the `MulticastSocket` class.

Example:

```
MulticastSocket socket = new MulticastSocket(4446);
```

Processes can join multicast groups and exchange messages efficiently.

## Conclusion

Multicast communication is an efficient one-to-many communication method where data is transmitted from a sender to a selected group of receivers simultaneously. It reduces network traffic, saves bandwidth, and is widely used in distributed systems and real-time applications.

### ➤ Message-oriented Communication

#### Message-Oriented Communication

**Message-oriented communication** is a type of Inter-Process Communication (IPC) in which processes communicate by sending and receiving discrete messages.

Instead of sharing memory directly, processes exchange information in the form of messages through communication channels provided by the operating system or network.

#### Basic Concept

Process A → Message → Process B

The sender creates a message, and the receiver reads that message.

#### Characteristics of Message-Oriented Communication

- Communication occurs through messages
- Processes remain independent
- No shared memory required
- Suitable for distributed systems
- Supports synchronization between processes

## Components of Message Communication

### 1. Sender

The process that creates and sends the message.

### 2. Receiver

The process that receives and processes the message.

### 3. Message

The unit of communication containing:

- Data
- Control information
- Headers

### 4. Communication Channel

Medium used for transmission:

- Pipes
- Message queues
- Sockets
- Mailboxes

## Types of Message-Oriented Communication

### 1. Direct Communication

Processes communicate directly by naming each other.

Sender → Receiver

### Example

```
send(P, message)
receive(Q, message)
```

## 2. Indirect Communication

Messages are exchanged through mailboxes or message queues.

Sender → Mailbox → Receiver

### Synchronous and Asynchronous Messaging

#### Synchronous (Blocking)

- Sender or receiver waits until communication completes.

#### Asynchronous (Non-Blocking)

- Processes continue execution immediately after sending/receiving.

### Message Passing Operations

#### Send Operation

Used to transmit messages.

```
send(destination, message)
```

#### Receive Operation

Used to receive messages.

```
receive(source, message)
```

### Advantages of Message-Oriented Communication

#### 1. Simplicity

Easy to implement and understand.

## 2. Process Isolation

Processes do not access each other's memory directly.

## 3. Better Security

Memory corruption risks are lower.

## 4. Suitable for Distributed Systems

Works across different machines connected through networks.

## 5. Synchronization Support

Helps coordinate process execution.

## Disadvantages

- Slower than shared memory
- Message copying overhead
- Kernel involvement increases latency
- Communication complexity for large systems

## Examples of Message-Oriented IPC Mechanisms

- Pipes
- Message queues
- Sockets
- Mailboxes
- Remote Procedure Calls (RPC)

## Applications

- Client-server systems
- Distributed computing
- Network communication

- Chat applications
- Real-time systems

## Message-Oriented vs Shared Memory Communication

| Feature              | Message-Oriented | Shared Memory    |
|----------------------|------------------|------------------|
| Communication Method | Messages         | Shared data area |
| Speed                | Moderate         | Very Fast        |
| Synchronization      | Easier           | More difficult   |
| Memory Sharing       | Not required     | Required         |
| Safety               | Higher           | Lower            |

## Conclusion

Message-oriented communication is an IPC method where processes exchange information through messages instead of shared memory. It provides better isolation, flexibility, and support for distributed systems, making it widely used in operating systems and network-based applications.

### ➤ MPI

**Nov-Dec 2023**

Q2) b) What is MPI? Describe the point-to-point communication in MPI with suitable diagram. Explain in brief, the following send operations in MPI : i) MPI\_Ssend ii) MPI\_Bsend iii) MPI\_Rsend iv) MPI\_Isend

Ans. MPI (Message Passing Interface)

**MPI (Message Passing Interface)** is a standardized library used for communication between processes in parallel and distributed computing systems.

It allows multiple processes running on one or more computers to exchange data efficiently through message passing.

MPI is widely used in:

- High-performance computing (HPC)

- Scientific simulations
- Parallel processing
- Distributed systems

## Features of MPI

- Portable and standardized
- Supports parallel computation
- Efficient communication
- Scalable for large systems
- Supports synchronous and asynchronous communication

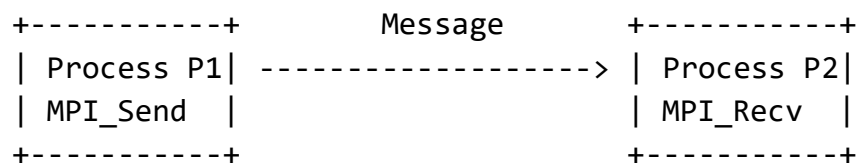
## Point-to-Point Communication in MPI

Point-to-point communication is a communication method where one process sends data directly to another specific process.

It mainly uses:

- `MPI_Send()`
- `MPI_Recv()`

## Basic Communication Diagram



- Sender process transmits the message.
- Receiver process accepts the message.

## MPI Send and Receive Syntax

### Send Operation

`MPI_Send(buffer, count, datatype, destination, tag, communicator)`

### Receive Operation

`MPI_Recv(buffer, count, datatype, source, tag, communicator, status)`

## Types of Send Operations in MPI

MPI provides several send modes depending on synchronization and buffering requirements.

### i) MPI\_Ssend (Synchronous Send)

`MPI_Ssend()` performs synchronous communication.

The sender waits until the receiver starts receiving the message.

#### Characteristics

- Blocking send
- Ensures synchronization
- Sender waits for receiver readiness

#### Diagram

Sender waits  $\longrightarrow$  Receiver starts receive

#### Syntax

`MPI_Ssend(buffer, count, datatype, dest, tag, comm)`

#### Advantages

- Safe communication
- Prevents buffer overflow

### Disadvantages

- Slower due to waiting

### ii) MPI\_Bsend (Buffered Send)

MPI\_Bsend() uses a user-provided buffer for sending messages.

The message is copied into the buffer, allowing the sender to continue execution immediately.

### Characteristics

- Buffered communication
- Sender does not wait after buffering

### Diagram

Sender → Buffer → Receiver

### Syntax

MPI\_Bsend(buffer, count, datatype, dest, tag, comm)

### Advantages

- Better performance
- Reduced blocking

### Disadvantages

- Extra memory required for buffers

### iii) MPI\_Rsend (Ready Send)

MPI\_Rsend() sends data only if the receiver is already prepared to receive it.

If the receiver is not ready, behavior becomes undefined.

### Characteristics

- Receiver must already call receive
- Faster communication

## Diagram

Receiver Ready ← Sender Sends

## Syntax

```
MPI_Rsend(buffer, count, datatype, dest, tag, comm)
```

## Advantages

- High performance
- Lower overhead

## Disadvantages

- Unsafe if receiver is not ready

## iv) MPI\_Isend (Immediate / Non-Blocking Send)

MPI\_Isend( ) starts sending data and immediately returns control to the sender without waiting for completion.

Communication continues in the background.

## Characteristics

- Non-blocking communication
- Enables parallel computation and communication

## Diagram

Sender starts send → Continues execution

## Syntax

```
MPI_Isend(buffer, count, datatype, dest, tag, comm, request)
```

## Advantages

- Improved parallelism
- Better CPU utilization

## Disadvantages

- More complex synchronization

## Comparison of MPI Send Operations

| Operation | Type         | Blocking  | Buffer Used | Receiver Must Be Ready |
|-----------|--------------|-----------|-------------|------------------------|
| MPI_Ssend | Synchronous  | Yes       | No          | Yes                    |
| MPI_Bsend | Buffered     | Partially | Yes         | No                     |
| MPI_Rsend | Ready Send   | No        | No          | Yes                    |
| MPI_Isend | Non-blocking | No        | Optional    | No                     |

## Applications of MPI

- Scientific computing
- Weather forecasting
- Image processing
- Machine learning
- Parallel simulations

## Conclusion

MPI is a standard communication library for parallel computing systems. Point-to-point communication allows direct data transfer between processes using send and receive operations. MPI provides different send modes such as MPI\_Ssend, MPI\_Bsend, MPI\_Rsend, and MPI\_Isend to support synchronization, buffering, and non-blocking communication requirements efficiently.

**Nov-Dec 2024**

**May-Jun 2025**

Q1) b) What is the purpose of Message Passing Interface? Illustrate the architectural model for MPI using send and receive primitives.

Ans. Purpose of Message Passing Interface (MPI)

**MPI (Message Passing Interface)** is a standardized communication protocol used for parallel and distributed computing. Its main purpose is to enable multiple processes running on one or more computers to communicate and exchange data efficiently.

MPI provides a set of communication functions that help processes cooperate while solving large computational problems.

## Purposes of MPI

### 1. Parallel Processing

MPI allows tasks to be divided among multiple processors to execute simultaneously.

### 2. Process Communication

Processes can exchange information using message passing.

### 3. Distributed Computing

MPI supports communication between processes running on different machines connected through a network.

### 4. High Performance Computing (HPC)

MPI is widely used in scientific and engineering applications requiring high computational power.

### 5. Scalability

MPI programs can run efficiently on:

- Small clusters
- Supercomputers
- Large distributed systems

### 6. Synchronization

MPI provides synchronization mechanisms between processes.

## Features of MPI

- Portable and standardized
- Supports point-to-point and collective communication
- Efficient communication
- Supports synchronous and asynchronous operations
- Language independent (C, C++, Fortran, etc.)

## Architectural Model of MPI

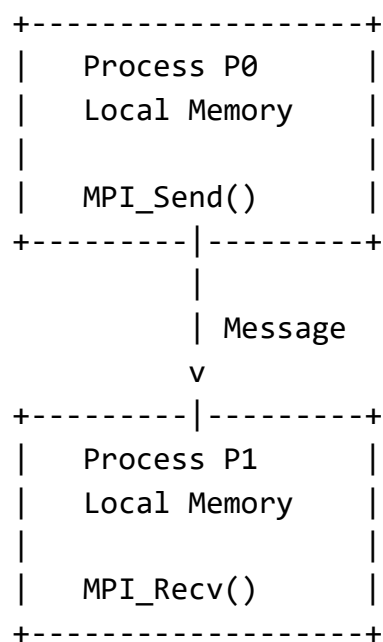
MPI follows a **distributed memory architecture**, where:

- Each process has its own local memory
- Processes communicate by sending and receiving messages

Communication mainly occurs using:

- `MPI_Send()`
- `MPI_Recv()`

## MPI Architectural Model Using Send and Receive Primitives



## Explanation of the Architecture

### 1. Independent Processes

Each MPI process executes independently with its own address space.

### 2. Message Passing

Data transfer occurs through messages instead of shared memory.

### 3. Send Primitive

The sender process uses:

`MPI_Send()`

to send data to another process.

### 4. Receive Primitive

The receiver process uses:

`MPI_Recv()`

to receive data.

## Working of Send and Receive

### Step 1

Sender process prepares data.

### Step 2

`MPI_Send()` transmits the message.

### Step 3

Message travels through communication network.

#### Step 4

Receiver process accepts message using `MPI_Recv()`.

#### Example Flow

Process P0:

`MPI_Send(data)`

↓

Communication Network

↓

Process P1:

`MPI_Recv(data)`

#### Advantages of MPI Architecture

- Efficient parallel execution
- High scalability
- Suitable for distributed systems
- Better fault isolation
- Supports heterogeneous systems

#### Disadvantages

- Programming complexity
- Explicit communication required
- Synchronization overhead

#### Applications of MPI

- Weather forecasting

- Scientific simulations
- Artificial intelligence
- Image processing
- Computational biology
- Parallel numerical analysis

## Conclusion

MPI is a standard message-passing system used for parallel and distributed computing. Its main purpose is to enable efficient communication and coordination among processes. The MPI architectural model uses distributed memory and communication primitives like `MPI_Send()` and `MPI_Recv()` for exchanging messages between processes.

## ➤ Network Virtualization

**May-Jun 2023**

Q1) b) What is Network virtualization? What are the types of overlay networks? What are the various advantages of overlay networks?

Ans. Network Virtualization

**Network virtualization** is the process of creating a virtual version of a physical network. It combines hardware and software network resources into logical, software-based networks that can operate independently from the underlying physical infrastructure.

It allows multiple virtual networks to run on the same physical network hardware.

Network virtualization is widely used in:

- Cloud computing
- Data centers
- Software-defined networking (SDN)
- Virtual machines and containers

## Objectives of Network Virtualization

- Efficient resource utilization
- Improved scalability

- Network flexibility
- Simplified management
- Isolation between networks

### Working of Network Virtualization

The physical network resources such as:

- Switches
- Routers
- Links
- Firewalls

are abstracted into virtual components.

These virtual networks behave like real networks but are software-controlled.

### Types of Network Virtualization

#### 1. Internal Network Virtualization

Virtual networks are created within a single physical system using:

- Virtual switches
- VLANs
- Hypervisors

#### Example

Communication between virtual machines inside one server.

#### 2. External Network Virtualization

Combines multiple physical networks into a single virtual network.

#### Example

Virtual private networks (VPNs).

## Overlay Networks

An **overlay network** is a virtual network built on top of another physical network.

In overlay networking:

- Virtual nodes connect through logical links
- Actual communication occurs through the underlying physical network

## Types of Overlay Networks

### 1. Peer-to-Peer (P2P) Overlay Network

Nodes communicate directly with each other without centralized control.

#### Features

- Distributed architecture
- Dynamic node participation

#### Examples

- BitTorrent
- Skype (older architecture)

### 2. Structured Overlay Network

Uses organized topology and routing algorithms.

#### Features

- Efficient data lookup
- Predictable routing

#### Example

Distributed Hash Tables (DHTs)

### 3. Unstructured Overlay Network

Connections between nodes are random.

#### Features

- Simple implementation
- Flexible topology

## Example

Gnutella

## 4. Tunneling-Based Overlay Network

Creates virtual tunnels over physical networks.

### Examples

- VPN
- GRE tunnels
- VXLAN

## 5. Application Layer Overlay Network

Built at the application layer for specific services.

### Examples

- Content Delivery Networks (CDNs)
- Streaming services

## Basic Overlay Network Diagram

Virtual Network (Overlay)

Node A      -----      Node B

    \                      /  
    \                      /

Underlying Physical Network

## Advantages of Overlay Networks

### 1. Scalability

Overlay networks can grow easily without changing physical infrastructure.

## 2. Flexibility

Virtual topologies can be modified dynamically.

## 3. Improved Resource Utilization

Physical resources are shared efficiently among multiple virtual networks.

## 4. Isolation and Security

Different virtual networks remain isolated from each other.

## 5. Simplified Network Management

Centralized software control simplifies configuration and monitoring.

## 6. Fault Tolerance

Overlay routing can bypass failed nodes or links.

## 7. Easy Deployment

New services and virtual networks can be created quickly.

## 8. Support for Cloud Computing

Overlay networks are essential for virtual machines, containers, and cloud platforms.

## Disadvantages of Overlay Networks

- Additional processing overhead
- Increased latency
- Complex routing mechanisms
- Security management challenges

## Applications of Overlay Networks

- Cloud computing
- VPNs
- Peer-to-peer systems
- Data centers
- Content delivery networks
- Software-defined networking

## Conclusion

Network virtualization creates software-based virtual networks over physical infrastructure. Overlay networks are virtual networks built on top of existing networks and can be structured, unstructured, peer-to-peer, tunneling-based, or application-layer overlays. Their major advantages include scalability, flexibility, security, simplified management, and efficient resource utilization.

### ➤ Overlay Networks

**May\_Jun\_2024**

Q1) b) Describe the overlay network and its benefits? Explain it with the help of Skype overlay architecture.

Ans. Overlay Network

An **overlay network** is a virtual network built on top of an existing physical network. In an overlay network, nodes are connected through logical or virtual links, while the actual communication takes place through the underlying physical network such as the Internet.

The overlay network creates an additional layer of communication that is independent of the physical infrastructure.

## Basic Concept of Overlay Network

Overlay Network (Virtual Links)

Node A ----- Node B  
  \                  /  
      \              /

\ /  
Underlying Physical Network

The virtual links may correspond to multiple physical connections.

### Characteristics of Overlay Networks

- Logical network over physical infrastructure
- Independent routing mechanisms
- Dynamic topology
- Scalable and flexible
- Supports distributed communication

### Benefits of Overlay Networks

#### 1. Scalability

Overlay networks can support a large number of nodes without modifying the physical network.

#### 2. Flexibility

Virtual connections can be created and changed dynamically.

#### 3. Efficient Resource Utilization

Multiple virtual networks can share the same physical infrastructure.

#### 4. Fault Tolerance

Alternative paths can be used if some nodes or links fail.

#### 5. Simplified Deployment

New services and applications can be added without changing hardware.

## 6. Improved Communication

Optimized routing and data delivery improve performance.

## 7. Support for Distributed Systems

Widely used in peer-to-peer systems, cloud computing, and streaming applications.

## Skype Overlay Architecture

Skype used an overlay peer-to-peer architecture to provide voice and video communication services over the Internet.

Skype's overlay network was built on top of the existing Internet infrastructure.

## Components of Skype Overlay Architecture

### 1. Ordinary Hosts (OH)

These are normal Skype users' systems.

#### Functions

- Voice/video communication
- Chat messaging
- File transfer

### 2. Super Nodes (SN)

Powerful nodes with:

- High bandwidth
- Public IP addresses
- Good processing power

These nodes acted as routers in the overlay network.

#### Functions

- Maintain routing information

- Help users locate other users
- Relay communication

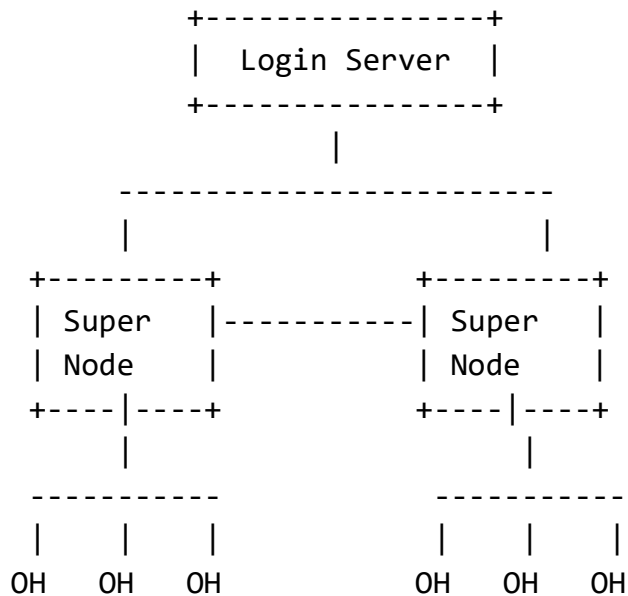
### 3. Login Server

Centralized server used only for:

- User authentication
- Login verification

After login, communication occurred mainly through the overlay network.

### Skype Overlay Architecture Diagram



### Working of Skype Overlay Network

#### Step 1: Login

User authenticates through the login server.

#### Step 2: Super Node Connection

The ordinary host connects to a nearby super node.

### Step 3: User Search

Super nodes help locate the destination user.

### Step 4: Communication

Voice/data packets are exchanged through:

- Direct peer-to-peer connection, or
- Relayed through super nodes

### Benefits of Skype Overlay Architecture

#### 1. Reduced Server Load

Most communication occurs directly between peers.

#### 2. Scalability

Millions of users can communicate simultaneously.

#### 3. Fault Tolerance

If one super node fails, another can take over.

#### 4. Efficient Routing

Super nodes optimize communication paths.

#### 5. Lower Infrastructure Cost

Less dependence on centralized servers.

### Disadvantages

- Complex management

- Security concerns
- Dependence on super nodes
- NAT/firewall traversal challenges

### Applications of Overlay Networks

- Voice over IP (VoIP)
- Peer-to-peer systems
- Content delivery networks
- Cloud computing
- Distributed applications

### Conclusion

An overlay network is a virtual communication network built over a physical network infrastructure. It provides scalability, flexibility, fault tolerance, and efficient communication. Skype used a peer-to-peer overlay architecture consisting of ordinary hosts, super nodes, and login servers to provide scalable and efficient voice and video communication services over the Internet.

### Nov\_Dec\_2024

Q2) a) Discuss the purpose of overlay network. Describe in brief, any three types of Overlay.

Ans. Purpose of Overlay Network

An **overlay network** is a virtual network built on top of an existing physical network. The nodes in the overlay network are connected through logical links, while the actual communication takes place through the underlying physical infrastructure such as the Internet.

Overlay networks add flexibility and advanced functionality without modifying the physical network.

## Purposes of Overlay Network

### 1. Scalability

Overlay networks can support a large number of nodes and users efficiently.

### 2. Flexibility

Virtual topologies and communication paths can be created or modified dynamically.

### 3. Efficient Resource Utilization

Multiple virtual networks can share the same physical infrastructure.

### 4. Improved Routing

Overlay networks can optimize routing and communication paths.

### 5. Fault Tolerance

Alternative routes can be used if some nodes or links fail.

### 6. Simplified Deployment

New services and applications can be implemented without changing the underlying hardware network.

### 7. Support for Distributed Applications

Used in:

- Peer-to-peer systems
- Cloud computing
- Streaming services
- Virtual private networks

## Types of Overlay Networks

There are several types of overlay networks. Three important types are:

### 1. Peer-to-Peer (P2P) Overlay Network

In a peer-to-peer overlay, nodes communicate directly with each other without relying heavily on centralized servers.

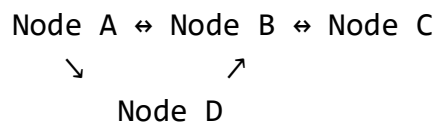
Each node can act as:

- Client
- Server

#### Features

- Distributed architecture
- Dynamic participation
- Resource sharing among peers

#### Diagram



#### Examples

- BitTorrent
- Early Skype architecture

#### Advantages

- Scalable
- Cost-effective
- Fault tolerant

### 2. Structured Overlay Network

Structured overlays organize nodes using predefined algorithms and topologies.

Data placement and routing follow strict rules.

## Features

- Efficient searching
- Predictable routing
- Organized structure

## Example

Distributed Hash Tables (DHT)

## Diagram

Node 1 → Node 2 → Node 3 → Node 4

## Advantages

- Fast lookup
- Better performance
- Efficient resource discovery

## 3. Unstructured Overlay Network

In unstructured overlays, nodes connect randomly without a fixed structure.

## Features

- Flexible topology
- Easy to build
- No strict organization

## Diagram

Node A ↔ Node B

↓                      ↓

Node C ↔ Node D

## Examples

- Gnutella

## Advantages

- Simple implementation
- Flexible node addition/removal

## Disadvantages

- Inefficient searching
- Higher network traffic

## Comparison of Overlay Types

### Comparison of Overlay Types

| Overlay Type         | Structure       | Routing Efficiency | Complexity |
|----------------------|-----------------|--------------------|------------|
| Peer-to-Peer         | Semi-structured | Moderate           | Moderate   |
| Structured Overlay   | Organized       | High               | High       |
| Unstructured Overlay | Random          | Lower              | Low        |

## Applications of Overlay Networks

- Cloud computing
- VPNs
- Peer-to-peer systems
- Streaming platforms
- Distributed storage systems

## Conclusion

The purpose of an overlay network is to provide scalable, flexible, and efficient communication over existing physical networks. Common types of overlays include peer-to-peer overlays, structured overlays, and unstructured overlays, each having different architectures, routing methods, and applications.

### ◆ Coordination

### ➤ Clock Synchronization

May\_Jun\_2025

Q1) a) Discuss the problem of clock synchronization in distributed operating systems. Illustrate the Berkeley's clock synchronization algorithm with neat diagram and the drawback of Network Time Protocol (NTP).

Ans. Clock Synchronization in Distributed Operating Systems

In a distributed operating system, multiple computers (nodes) work together and communicate through a network. Each system has its own local clock, but these clocks may not show exactly the same time because of:

- Clock drift
- Different clock speeds
- Network delays
- Hardware differences

As a result, clocks become unsynchronized over time.

Clock synchronization is necessary to maintain a common notion of time across distributed systems.

### Problems Due to Unsynchronized Clocks

If clocks are not synchronized, several problems may occur:

#### 1. Incorrect Event Ordering

Distributed systems may fail to determine which event occurred first.

#### 2. File Consistency Problems

File modification times may become incorrect.

#### 3. Database Inconsistency

Transactions may appear in the wrong order.

#### 4. Coordination Issues

Processes relying on timing may fail.

## 5. Security Problems

Authentication systems using timestamps may malfunction.

### Need for Clock Synchronization

Clock synchronization helps in:

- Maintaining event ordering
- Coordinating distributed processes
- Logging and debugging
- Distributed transaction management
- Resource sharing

### Berkeley's Clock Synchronization Algorithm

The **Berkeley Algorithm** is used for internal clock synchronization in distributed systems where no machine has an accurate external time source.

A master system periodically synchronizes all slave clocks.

### Working of Berkeley Algorithm

#### Step 1: Master Polls Slaves

The master node requests current time from all slave nodes.

#### Step 2: Slaves Reply

Each slave sends its local clock value to the master.

#### Step 3: Average Time Calculation

The master:

- Calculates clock differences
- Ignores faulty clocks if needed
- Computes average time

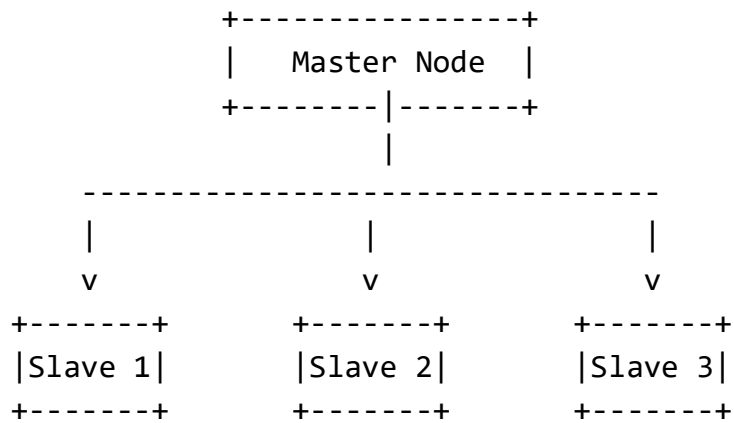
#### Step 4: Time Adjustment

The master sends adjustment values to all slaves.

#### Step 5: Clocks are Updated

Each node adjusts its clock gradually.

#### Berkeley Algorithm Diagram



1. Master polls slaves
2. Slaves send clock times
3. Master computes average
4. Adjustment values sent back

#### Example

Suppose clocks show:

| System  | Time  |
|---------|-------|
| Master  | 10:00 |
| Slave 1 | 10:05 |
| Slave 2 | 09:58 |
| Slave 3 | 10:02 |

Average time is calculated, and each system adjusts accordingly.

#### Advantages of Berkeley Algorithm

- Does not require external UTC source
- Suitable for LAN environments
- Handles faulty clocks
- Provides internal synchronization

#### Disadvantages

- Master node failure affects synchronization
- Network delay may reduce accuracy
- Scalability issues in large systems

#### Network Time Protocol (NTP)

**Network Time Protocol (NTP)** is a protocol used to synchronize clocks of computers over networks using UTC reference servers.

It uses a hierarchical structure of time servers called strata.

## Drawbacks of NTP

### 1. Network Delay Sensitivity

Variable network delays reduce synchronization accuracy.

### 2. Security Vulnerabilities

NTP servers may suffer from:

- Spoofing
- DDoS attacks
- False time injection

### 3. Dependence on External Servers

Synchronization depends on availability of external time servers.

### 4. Complexity

NTP configuration and management can be complex.

### 5. Reduced Accuracy in Large Networks

Accuracy decreases in highly congested or wide-area networks.

### 6. Clock Drift Between Synchronizations

Local clocks may drift between update intervals.

## Comparison: Berkeley Algorithm vs NTP

| Feature             | Berkeley Algorithm | NTP                     |
|---------------------|--------------------|-------------------------|
| Time Source         | Internal average   | External UTC server     |
| Suitable For        | LAN systems        | Internet/global systems |
| Accuracy            | Moderate           | High                    |
| Central Control     | Master node        | Hierarchical servers    |
| External Dependency | No                 | Yes                     |

## Conclusion

Clock synchronization is essential in distributed operating systems to maintain consistency and proper event ordering. The Berkeley Algorithm synchronizes clocks internally using a master-slave approach and average time calculation. NTP synchronizes systems using external UTC servers but suffers from issues like network delays, security vulnerabilities, and dependency on external servers.

## Nov\_Dec\_2023

Q1) a) Why is computer clock synchronization necessary? Describe the design requirements for a system to synchronize the clocks in a distributed system.

Ans. Need for Computer Clock Synchronization

In distributed systems, multiple computers operate independently, and each system has its own local clock. These clocks may run at slightly different speeds due to hardware differences and clock drift, causing them to show different times.

Clock synchronization is necessary to maintain a consistent notion of time across all systems in the distributed environment.

## Why Clock Synchronization is Necessary

### 1. Correct Event Ordering

Distributed systems need to determine the exact sequence of events.

Without synchronization:

- Events may appear in the wrong order.

#### Example

System A sends message at 10:00

System B receives at 09:59

This creates inconsistency.

## 2. Coordination Between Processes

Many distributed applications depend on synchronized timing for proper coordination.

Examples:

- Distributed databases
- Parallel processing
- Transaction systems

## 3. File Consistency

File timestamps must remain accurate across multiple systems.

## 4. Security and Authentication

Authentication systems often use timestamps to validate requests.

Unsynchronized clocks may:

- Reject valid users
- Accept expired credentials

## 5. Logging and Debugging

System logs from different machines must have consistent timestamps for troubleshooting.

## 6. Resource Sharing

Shared resources require synchronized timing for proper access control and scheduling.

### Design Requirements for Clock Synchronization in Distributed Systems

A clock synchronization system should satisfy several important design requirements.

#### 1. Accuracy

The synchronization system should minimize time differences between clocks.

Clock difference  $\approx$  minimum

Higher accuracy improves system consistency.

#### 2. Reliability

The system should continue functioning even if:

- Some nodes fail
- Messages are delayed
- Network problems occur

#### 3. Fault Tolerance

Synchronization should tolerate:

- Faulty clocks
- Crashed nodes
- Incorrect time values

Faulty systems should not affect the entire network.

#### 4. Scalability

The synchronization mechanism should work efficiently for:

- Small networks
- Large distributed systems
- Cloud environments

## 5. Low Communication Overhead

Synchronization should minimize:

- Network traffic
- CPU usage
- Message exchanges

Excessive synchronization messages reduce performance.

## 6. Consistency

All clocks should maintain approximately the same time throughout the system.

## 7. Adaptability to Network Delays

The system should account for:

- Variable transmission delays
- Packet loss
- Congestion

## 8. Security

The synchronization protocol must protect against:

- Fake time servers
- Spoofing attacks
- Unauthorized modifications

## 9. Convergence

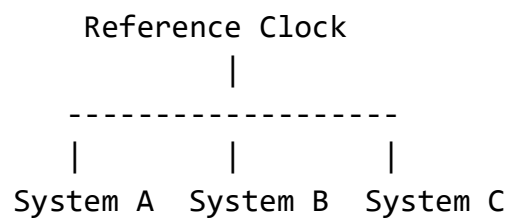
All clocks should gradually converge toward a common synchronized time.

## 10. External Time Reference Support

Some systems require synchronization with global standard time such as:

- UTC
- GPS clocks
- Atomic clocks

### Basic Clock Synchronization Model



The reference clock distributes synchronized time to all systems.

### Challenges in Clock Synchronization

- Clock drift
- Variable network delays
- Different hardware speeds
- Node failures
- Scalability issues

### Examples of Synchronization Protocols

- Cristian's Algorithm
- Berkeley Algorithm
- Network Time Protocol (NTP)

## Conclusion

Computer clock synchronization is necessary in distributed systems to maintain consistency, correct event ordering, coordination, security, and reliable communication. A good synchronization system should provide high accuracy, reliability, fault tolerance, scalability, low overhead, and protection against network and security issues.

Q1) b) What is NTP? With the help of a diagram, describe how NTP works.

Ans. NTP (Network Time Protocol)

**Network Time Protocol (NTP)** is a protocol used to synchronize the clocks of computers over a network. It ensures that all systems in a distributed environment maintain nearly the same accurate time.

NTP synchronizes computer clocks with highly accurate reference clocks such as:

- Atomic clocks
- GPS clocks
- UTC time servers

It is widely used in:

- Distributed systems
- Servers
- Cloud computing
- Banking systems
- Communication networks

## Need for NTP

Different computer clocks drift over time due to hardware differences and environmental conditions. NTP helps:

- Maintain accurate system time
- Synchronize distributed systems
- Ensure proper event ordering
- Support secure authentication
- Maintain accurate logs

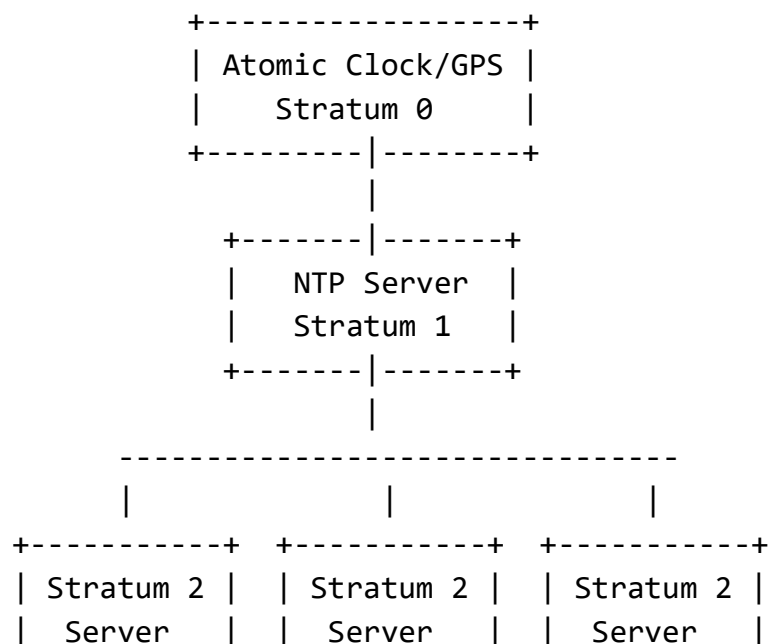
## NTP Architecture

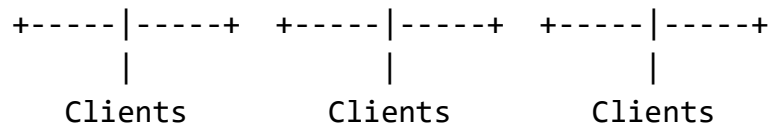
NTP follows a hierarchical structure called **Stratum Levels**.

### Stratum Levels

| Stratum    | Description                                          |
|------------|------------------------------------------------------|
| Stratum 0  | Highly accurate reference clocks (GPS, atomic clock) |
| Stratum 1  | Primary time servers directly connected to Stratum 0 |
| Stratum 2  | Servers synchronized from Stratum 1                  |
| Stratum 3+ | Lower-level synchronized servers                     |

### NTP Working Diagram





## How NTP Works

### Step 1: Reference Time Source

A highly accurate clock (GPS or atomic clock) provides standard time.

### Step 2: Primary Server Synchronization

Stratum 1 servers synchronize directly with the reference clock.

### Step 3: Lower-Level Synchronization

Stratum 2 and lower servers synchronize with higher-level servers.

### Step 4: Client Synchronization

Client systems request time information from NTP servers.

### Step 5: Delay Calculation

NTP calculates:

- Network delay
- Clock offset
- Transmission time

### Step 6: Clock Adjustment

The client gradually adjusts its clock to match the server time.

## NTP Timestamp Exchange

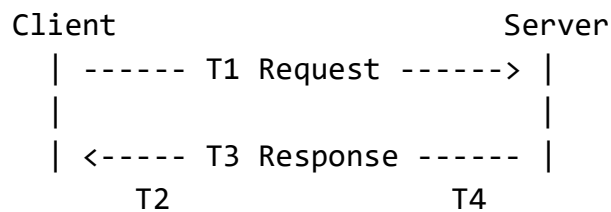
NTP uses four timestamps:

| Timestamp | Meaning                 |
|-----------|-------------------------|
| T1        | Client sends request    |
| T2        | Server receives request |
| T3        | Server sends reply      |
| T4        | Client receives reply   |

Using these timestamps, NTP computes:

- Round-trip delay
- Time offset

## NTP Communication Process



## Advantages of NTP

- High time accuracy
- Supports distributed systems
- Automatic synchronization
- Scalable hierarchical design
- Reliable over large networks

## Disadvantages of NTP

- Sensitive to network delays
- Security vulnerabilities
- Dependency on external servers
- Reduced accuracy in congested networks

## Applications of NTP

- Banking systems
- Cloud computing
- Distributed databases
- Communication systems
- Server synchronization
- Network management

## Conclusion

NTP is a protocol used to synchronize computer clocks over networks using a hierarchical structure of time servers. It works by exchanging timestamps between clients and servers to calculate clock offsets and network delays, helping distributed systems maintain accurate and consistent time.

Q2) a) Why is the Berkeley algorithm used? Describe how it works using pseudocode.

Ans. Berkeley Algorithm

The **Berkeley Algorithm** is a clock synchronization algorithm used in distributed systems to synchronize the clocks of multiple computers connected in a network.

Unlike protocols that use an external UTC source, the Berkeley algorithm performs **internal synchronization** by calculating an average time from all participating systems.

It is mainly suitable for:

- Local Area Networks (LANs)
- Distributed operating systems
- Systems without external time servers

## Why is the Berkeley Algorithm Used?

The Berkeley algorithm is used to solve the problem of clock drift in distributed systems where each computer maintains its own local clock.

## Purpose of Berkeley Algorithm

### 1. Synchronize System Clocks

Ensures all systems maintain approximately the same time.

### 2. Internal Time Synchronization

Works even when no external UTC or atomic clock is available.

### 3. Maintain Consistency

Helps maintain:

- Correct event ordering
- Accurate timestamps
- Coordination among distributed processes

### 4. Reduce Clock Drift

Minimizes differences between system clocks over time.

### 5. Improve Distributed System Reliability

Consistent timing improves communication and synchronization.

## Working of Berkeley Algorithm

The algorithm uses a **master-slave approach**.

One system acts as the **master**, while other systems act as **slaves**.

## Steps of Berkeley Algorithm

### Step 1: Master Polls Slaves

The master requests current clock values from all slave systems.

### Step 2: Slaves Send Their Time

Each slave replies with its local clock time.

### Step 3: Master Computes Average Time

The master:

- Calculates time differences
- Ignores faulty clocks if needed
- Computes average system time

### Step 4: Calculate Adjustments

The master calculates how much each clock should advance or retard.

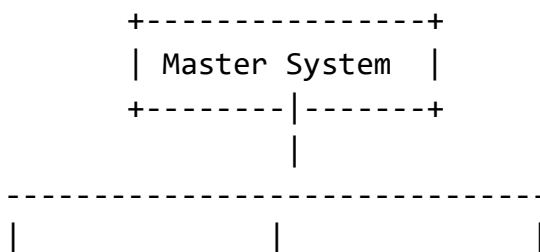
### Step 5: Send Corrections

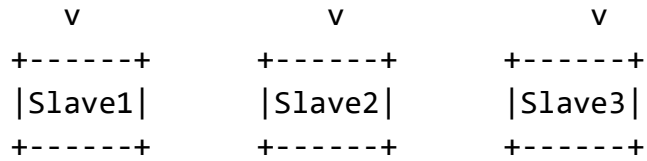
Adjustment values are sent to all slave systems.

### Step 6: Clocks are Updated

Each system gradually adjusts its clock.

## Berkeley Algorithm Diagram





1. Poll clocks
2. Receive times
3. Compute average
4. Send adjustments

### Pseudocode for Berkeley Algorithm

#### Master Process:

1. Send time request to all slave nodes
2. Receive clock values from all slaves
3. Calculate time difference for each slave
4. Ignore faulty or extreme values
5. Compute average time
6. Calculate adjustment for each node
7. Send adjustment values to slaves

#### Slave Process:

1. Receive request from master
2. Send local clock time to master
3. Receive adjustment value
4. Update local clock gradually

### Simplified Algorithm

`average_time = sum(all_clock_times) / number_of_nodes`

for each node:

`adjustment = average_time - local_clock`

send adjustment to node

### Example

Suppose:

| System  | Time  |
|---------|-------|
| Master  | 10:00 |
| Slave 1 | 10:03 |
| Slave 2 | 09:58 |
| Slave 3 | 10:01 |

Average time:

$$(10:00 + 10:03 + 09:58 + 10:01) / 4 \\ \approx 10:00:30$$

Each system adjusts its clock accordingly.

### Advantages of Berkeley Algorithm

- No external UTC source required
- Good for local distributed systems
- Handles faulty clocks
- Simple implementation

### Disadvantages

- Master node failure affects synchronization
- Less accurate over large networks
- Network delays can affect timing

## Applications

- Distributed operating systems
- LAN synchronization
- Cluster computing
- Internal enterprise networks

## Conclusion

The Berkeley algorithm is used for internal clock synchronization in distributed systems without requiring an external time source. It works by polling system clocks, computing an average time, and sending correction values to all nodes so that all clocks remain synchronized.

## Nov\_Dec\_2024

Q1) a) Why is clock synchronization using a global time impossible in a distributed system? Illustrate the active time server based centralized clock synchronization algorithm.

Ans. Why Global Time Synchronization is Impossible in Distributed Systems

In a distributed system, multiple computers communicate over a network, and each computer has its own local clock. Achieving perfect synchronization with a single global time is practically impossible because of several physical and network limitations.

## Reasons Why Global Clock Synchronization is Impossible

### 1. Clock Drift

Every computer clock runs at a slightly different speed due to:

- Hardware differences
- Temperature variations
- Crystal oscillator imperfections

As a result, clocks gradually drift apart over time.

## 2. Variable Network Delays

Messages exchanged between systems experience:

- Transmission delays
- Propagation delays
- Queuing delays
- Processing delays

These delays are unpredictable and vary continuously.

Sender -----(variable delay)-----> Receiver

Hence, exact time transfer is impossible.

## 3. Absence of a Perfect Global Clock

There is no universally accessible perfectly accurate clock directly available to all distributed systems simultaneously.

## 4. Asynchronous Nature of Distributed Systems

Distributed systems operate asynchronously:

- No common memory
- No shared clock
- Independent execution

Thus exact synchronization cannot be guaranteed.

## 5. Message Loss and Failures

Network failures, packet loss, and node crashes can interrupt synchronization.

## 6. Physical Limitations

Even signals traveling at the speed of light require finite time to move across networks.

Perfect simultaneity is physically impossible.

Therefore

Distributed systems aim for:

- Approximate synchronization
- Bounded clock differences
- Logical consistency

rather than perfectly identical clocks.

### Active Time Server Based Centralized Clock Synchronization Algorithm

This algorithm uses a centralized **time server** that actively synchronizes clocks of all client machines.

The server periodically sends the correct time to all clients.

### Working of Active Time Server Algorithm

#### Step 1: Time Server Maintains Correct Time

The centralized server synchronizes itself with:

- UTC
- Atomic clock
- GPS clock

#### Step 2: Server Periodically Sends Time Updates

The server actively broadcasts current time to all client systems.

#### Step 3: Clients Receive Time

Each client receives the synchronization message.

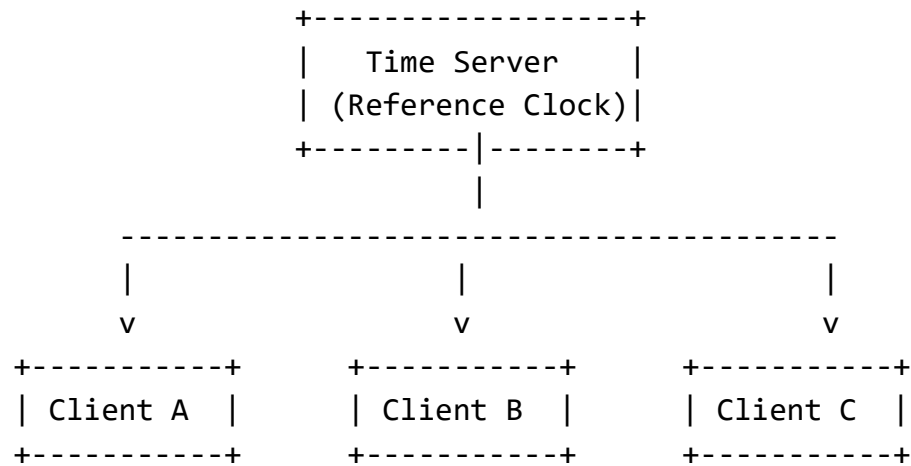
#### Step 4: Delay Compensation

Clients estimate network delay and adjust their clocks.

### Step 5: Clock Adjustment

Clients gradually correct their local clocks.

## Diagram of Active Time Server Based Synchronization



1. Server broadcasts current time
2. Clients receive and adjust clocks

## Algorithm Steps

1. Time server gets accurate reference time
2. Server sends current time to all clients
3. Clients calculate transmission delay
4. Clients adjust their clocks
5. Synchronization repeats periodically

### Example

Suppose:

- Server time = 10:00:00
- Network delay = 20 ms

Client estimates:

Adjusted time = 10:00:00 + 20 ms

and updates its clock accordingly.

## Advantages

### 1. Simple Implementation

Easy centralized management.

### 2. Good Accuracy

Provides reasonable synchronization in LANs.

### 3. Centralized Control

One server manages all clients.

## Disadvantages

### 1. Single Point of Failure

If the server crashes, synchronization stops.

### 2. Scalability Issues

Performance decreases with many clients.

### 3. Network Delay Problems

Variable delays reduce accuracy.

#### 4. Heavy Server Load

Central server may become overloaded.

#### Applications

- Local distributed systems
- Enterprise networks
- Cluster computing
- Internal LAN synchronization

#### Conclusion

Perfect global clock synchronization is impossible in distributed systems because of clock drift, unpredictable network delays, asynchronous execution, and physical limitations. The active time server based centralized synchronization algorithm addresses this problem by using a central server that periodically broadcasts time updates to all clients for approximate synchronization.

#### May\_Jun\_2023

Q2) a) Describe Lamport's clock algorithm and give an example of how to order events in a distributed system. What are the algorithm's limitations?

Ans. Lamport's Clock Algorithm

**Lamport's Logical Clock Algorithm** is a method used in distributed systems to determine the logical ordering of events when there is no global physical clock.

It was proposed by Leslie Lamport.

The algorithm provides a way to establish the **happened-before relationship** between events in distributed systems.

#### Need for Lamport's Clock

In distributed systems:

- Each process has its own local clock
- Perfect clock synchronization is impossible

- Events occur independently

Therefore, logical clocks are used to determine event ordering instead of exact physical time.

### Happened-Before Relationship

Lamport defined a relation called:

$a \rightarrow b$

which means:

Event **a happened before event b**

### Rules of Happened-Before Relation

#### 1. Same Process Rule

If two events occur in the same process, their execution order defines event order.

$a \text{ occurs before } b \Rightarrow a \rightarrow b$

#### 2. Message Passing Rule

If a message is sent by one process and received by another:

$\text{send}(m) \rightarrow \text{receive}(m)$

#### 3. Transitivity Rule

If:

$a \rightarrow b \text{ and } b \rightarrow c$

then:

$a \rightarrow c$

## Lamport Logical Clock Algorithm

Each process maintains a logical clock counter.

### Algorithm Rules

#### Rule 1: Increment Before Every Event

Before executing any event:

$$LC = LC + 1$$

#### Rule 2: Send Message

When sending a message:

- Increment clock
- Attach timestamp to the message

$$\text{message.timestamp} = LC$$

#### Rule 3: Receive Message

When receiving a message with timestamp T:

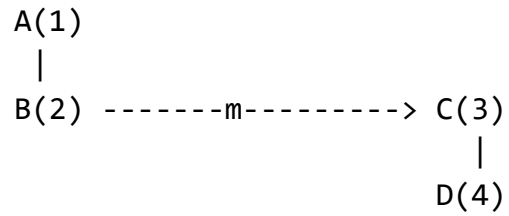
$$LC = \max(LC, T) + 1$$

### Example of Event Ordering

Consider two processes:

Process P1

Process P2



## Step-by-Step Explanation

### Process P1

| Event      | Clock Value |
|------------|-------------|
| A          | 1           |
| B (send m) | 2           |

Message carries timestamp = 2

### Process P2

Before receiving message:

- Local clock = 2

On receiving message timestamp 2:

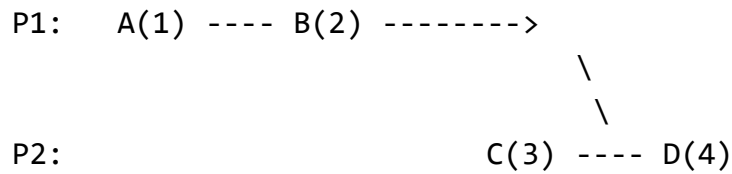
$$LC = \max(2, 2) + 1 = 3$$

| Event         | Clock Value |
|---------------|-------------|
| C (receive m) | 3           |
| D             | 4           |

## Ordered Events

A → B → C → D

## Diagram of Lamport Clock



## Advantages of Lamport's Clock

### 1. Simple Implementation

Easy to understand and implement.

### 2. Logical Event Ordering

Provides partial ordering of distributed events.

### 3. No Need for Physical Clock Synchronization

Works without synchronized hardware clocks.

### 4. Useful in Distributed Algorithms

Used in:

- Mutual exclusion
- Distributed databases
- Event synchronization

## Limitations of Lamport's Clock Algorithm

### 1. Cannot Detect Concurrent Events

If two events are unrelated, Lamport clocks may still assign an arbitrary order.

Example:

A || B

(concurrent events)

Lamport clocks cannot explicitly identify concurrency.

## 2. Logical Time Does Not Represent Real Time

Clock values only indicate ordering, not actual elapsed time.

## 3. Partial Ordering Only

The algorithm cannot always provide complete causal relationships.

## 4. Same Timestamp Possibility

Different events may receive identical logical times in some situations.

Tie-breaking mechanisms may be needed.

## Comparison with Vector Clocks

| Feature                  | Lamport Clock | Vector Clock           |
|--------------------------|---------------|------------------------|
| Event Ordering           | Partial       | Better causal ordering |
| Detect Concurrent Events | No            | Yes                    |
| Complexity               | Low           | Higher                 |

## Applications

- Distributed mutual exclusion
- Distributed databases
- Event synchronization
- Distributed debugging
- Message ordering systems

## Conclusion

Lamport's clock algorithm provides a logical mechanism for ordering events in distributed systems without requiring synchronized physical clocks. It works by assigning logical timestamps to events and maintaining happened-before relationships through message passing. However, it cannot identify concurrent events and does not represent actual physical time accurately.

### ➤ Logical Clocks

May\_Jun\_2024

Q2) a) What are the advantages of logical clock over physical clock? Consider Figure 1 that shows four processes (P1, P2, P3, P4) with events a, b, c, ... and messages communicating between them. Assume that initial logical clock values are all initialized to 0. List the Lamport timestamps for each event shown in Figure 1. Assume that each process maintains a logical clock as a single integer value as a Lamport clock. Provide timestamps for each labeled event.

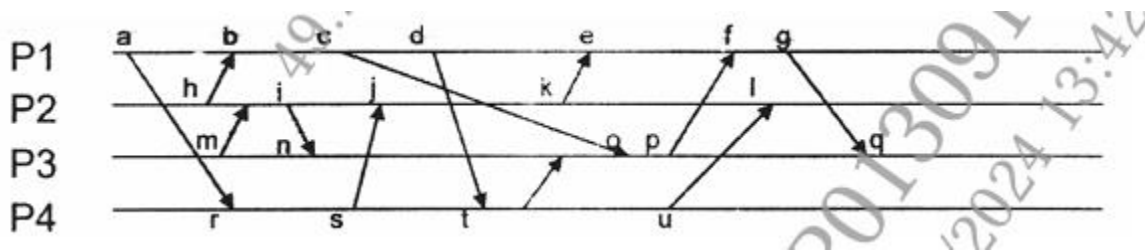


Figure 1 : Four Processes P1, P2, P3, P4 run events a, b, c, d ..... to send and receive messages

Ans. Advantages of Logical Clock over Physical Clock

Logical clocks are preferred in distributed systems because perfect physical clock synchronization is difficult to achieve.

### Advantages

#### 1. No Need for Global Time Synchronization

Logical clocks do not depend on synchronized hardware clocks or UTC time.

## 2. Avoids Clock Drift Problems

Physical clocks drift due to hardware imperfections, while logical clocks maintain consistent event ordering.

## 3. Correct Event Ordering

Logical clocks help determine the causal relationship between events using the “happened-before” relation.

## 4. Independent of Network Delays

They are less affected by variable communication delays.

## 5. Simpler Implementation

Logical clocks are implemented using simple integer counters.

## 6. Suitable for Distributed Systems

Distributed systems often lack shared memory and common clocks, making logical clocks more practical.

## Lamport Timestamp Calculation

### Message Relations from Figure

The message communications are:

$a \rightarrow r$

$m \rightarrow h$

$i \rightarrow n$

$s \rightarrow j$

$c \rightarrow o$

$d \rightarrow t$

$k \rightarrow e$

$t \rightarrow p$

$u \rightarrow l$

$p \rightarrow f$   
 $g \rightarrow q$

### Rules Used

For Internal/Send Event

$$LC = LC + 1$$

For Receive Event

$$LC = \max(LC, \text{received\_timestamp}) + 1$$

### Step-by-Step Lamport Timestamps

Process P1

| Event | Explanation       | Timestamp |
|-------|-------------------|-----------|
| a     | Send to r         | 1         |
| b     | Receive from m(1) | 2         |
| c     | Send to o         | 3         |
| d     | Send to t         | 4         |
| e     | Receive from k(5) | 6         |
| f     | Receive from p(6) | 7         |
| g     | Send to q         | 8         |

## Process P2

| Event | Explanation       | Timestamp |
|-------|-------------------|-----------|
| h     | Receive from m(1) | 2         |
| i     | Send to n         | 3         |
| j     | Receive from s(3) | 4         |
| k     | Send to e         | 5         |
| l     | Receive from u(6) | 7         |

## Process P3

| Event | Explanation       | Timestamp |
|-------|-------------------|-----------|
| m     | Send to h and b   | 1         |
| n     | Receive from i(3) | 4         |
| o     | Receive from c(3) | 5         |
| p     | Receive from t(5) | 6         |
| q     | Receive from g(8) | 9         |

## Process P4

| Event | Explanation                     | Timestamp |
|-------|---------------------------------|-----------|
| r     | Receive from a(1)               | 2         |
| s     | Send to j                       | 3         |
| t     | Receive from d(4) and send to p | 5         |
| u     | Send to l                       | 6         |

### Final Lamport Timestamps

P1 : a=1, b=2, c=3, d=4, e=6, f=7, g=8

P2 : h=2, i=3, j=4, k=5, l=7

P3 : m=1, n=4, o=5, p=6, q=9

P4 : r=2, s=3, t=5, u=6

### ➤ Mutual Exclusion

#### May\_Jun\_2024

Q2) b) What is the need of mutual exclusion? Explain the permission-based centralized distributed mutual exclusion algorithm.

Ans. Need of Mutual Exclusion

In distributed systems, multiple processes may try to access shared resources simultaneously such as:

- Shared files
- Databases
- Printers
- Memory
- Variables

If multiple processes access the same critical resource at the same time, it may lead to:

- Data inconsistency
- Race conditions
- Resource corruption
- Incorrect computation

**Mutual exclusion** ensures that only one process at a time enters the **critical section** and accesses the shared resource.

### Critical Section

A **critical section** is a part of a program where shared resources are accessed.

Process → Entry → Critical Section → Exit → Remainder

## Need for Mutual Exclusion

### 1. Prevent Race Conditions

Ensures orderly access to shared resources.

### 2. Maintain Data Consistency

Prevents simultaneous modifications of shared data.

### 3. Avoid Resource Conflicts

Ensures proper use of printers, files, databases, etc.

### 4. Synchronization

Coordinates process execution in distributed systems.

### 5. System Reliability

Prevents unexpected and inconsistent system behavior.

## Distributed Mutual Exclusion

In distributed systems:

- No shared memory exists
- Processes communicate using messages
- Synchronization becomes more difficult

Distributed mutual exclusion algorithms coordinate access using message passing.

## Permission-Based Centralized Distributed Mutual Exclusion Algorithm

This algorithm uses a **single coordinator (central server)** to control access to the critical section.

A process must obtain permission from the coordinator before entering the critical section.

### Components

#### 1. Coordinator Process

Controls access to the critical section.

#### 2. Client Processes

Request permission to enter the critical section.

### Working of the Algorithm

#### Step 1: Request

When a process wants to enter the critical section, it sends a request message to the coordinator.

REQUEST → Coordinator

#### Step 2: Permission Check

The coordinator checks whether another process is currently using the resource.

- If resource is free → Grant permission
- Otherwise → Queue the request

#### Step 3: Grant Permission

Coordinator sends a reply message.

GRANT → Process

The process enters the critical section.

#### Step 4: Release Resource

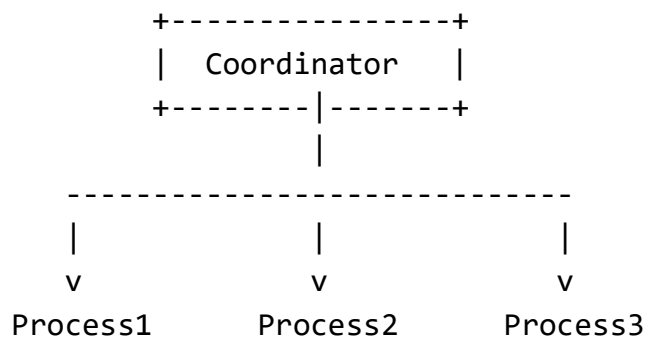
After completing execution, the process sends a release message.

RELEASE → Coordinator

#### Step 5: Next Process Selection

Coordinator grants permission to the next waiting process.

#### Diagram of Centralized Mutual Exclusion



1. Request sent to coordinator
2. Coordinator grants permission
3. Process enters critical section
4. Release message sent

## Algorithm Steps

### At Process $P_i$

1. Send REQUEST to coordinator
2. Wait for GRANT
3. Enter critical section
4. Execute critical section
5. Send RELEASE to coordinator

### At Coordinator

1. If resource free:  
    send GRANT  
    mark resource busy
2. Else:  
    queue request
3. On RELEASE:  
    select next process  
    send GRANT

## Message Complexity

For one critical section execution:

| Operation | Messages |
|-----------|----------|
| Request   | 1        |
| Grant     | 1        |
| Release   | 1        |

Total:

3 messages per critical section

## Advantages

### 1. Simple Implementation

Easy to design and manage.

### 2. Low Message Overhead

Requires only three messages.

### 3. Ensures Mutual Exclusion

Only one process enters the critical section.

## Disadvantages

### 1. Single Point of Failure

If coordinator crashes, the system stops.

### 2. Bottleneck Problem

Coordinator may become overloaded.

### 3. Poor Scalability

Performance decreases as number of processes increases.

## Applications

- Distributed databases
- File servers
- Printer sharing
- Resource management systems

## Conclusion

Mutual exclusion is required to prevent simultaneous access to shared resources in distributed systems. The permission-based centralized distributed mutual exclusion algorithm uses a coordinator process to grant permission for entering the critical section, ensuring safe and synchronized resource access.

## Nov\_Dec\_2024

Q2) b) What are the two important properties of token-based approach? Explain token-ring algorithm to achieve mutual exclusion in a distributed system.

Ans. Important Properties of Token-Based Approach

In token-based distributed mutual exclusion algorithms, a special message called a **token** gives permission to enter the critical section.

A process can enter the critical section only if it possesses the token.

## Two Important Properties of Token-Based Approach

### 1. Uniqueness of Token

At any time, there must be **exactly one token** in the system.

#### Reason

- Prevents multiple processes from entering the critical section simultaneously
- Ensures mutual exclusion

Only one token  $\Rightarrow$  Only one process in critical section

### 2. Token Circulation / Accessibility

The token must eventually reach every requesting process.

#### Reason

- Prevents starvation
- Ensures fairness
- Guarantees every process gets a chance to execute

Token circulates among all processes

### Token-Ring Algorithm for Distributed Mutual Exclusion

The **Token-Ring Algorithm** is a token-based distributed mutual exclusion algorithm where processes are arranged in a logical ring structure.

A unique token continuously circulates around the ring.

A process can enter the critical section only when it receives the token.

### Structure of Token Ring

Processes are connected logically in a circular manner.

P1 → P2 → P3 → P4 → P1

### Working of Token-Ring Algorithm

#### Step 1: Logical Ring Formation

All processes are organized into a logical ring.

Each process knows:

- Its successor
- Its predecessor

#### Step 2: Token Circulation

A unique token continuously moves from one process to the next.

#### Step 3: Request for Critical Section

When a process wants to enter the critical section:

- It waits for the token.

#### Step 4: Token Reception

When the token arrives:

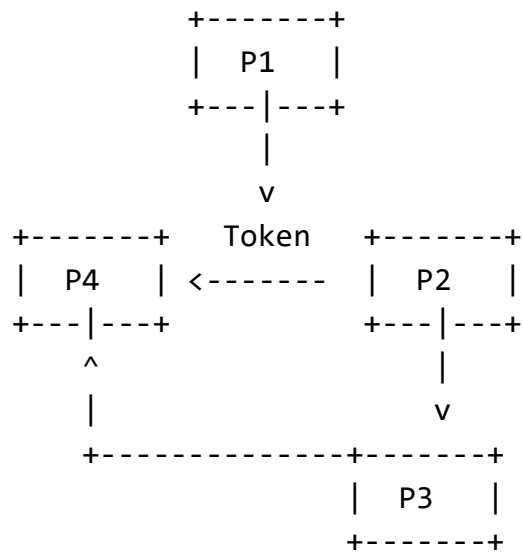
- If the process needs the critical section → enter CS
- Otherwise → immediately pass token to next process

#### Step 5: Release Critical Section

After completing execution:

- Process forwards token to next process in ring

#### Token Ring Diagram



## Algorithm Steps

### For Each Process $P_i$

1. Wait for token
2. If critical section required:  
    enter critical section
3. Execute critical section
4. Pass token to successor

## Example

Suppose:

- $P_3$  wants to enter critical section
- Token currently at  $P_1$

Flow:

$P_1 \rightarrow P_2 \rightarrow P_3$

$P_3$  receives token and enters critical section.

After execution:

$P_3 \rightarrow P_4$

Token continues circulating.

## Advantages of Token-Ring Algorithm

### 1. Guaranteed Mutual Exclusion

Only token holder enters critical section.

### 2. No Starvation

Every process eventually receives token.

### 3. Fairness

Processes get access in ring order.

### 4. No Deadlock

System continues as long as token exists.

## Disadvantages

#### 1. Token Loss Problem

If token is lost, system stops.

#### 2. Ring Failure Problem

Failure of one process can break the ring.

#### 3. Delay in Large Systems

Processes may wait long for token circulation.

#### 4. Token Recovery Complexity

Generating duplicate-free replacement token is difficult.

## Message Complexity

Only:

1 token message transfer

is needed between neighboring processes.

## Applications

- Distributed databases
- Distributed operating systems
- Resource allocation systems
- Network synchronization

## Conclusion

The token-based approach relies on two key properties: uniqueness of the token and guaranteed token circulation. The Token-Ring Algorithm achieves distributed mutual exclusion by passing a unique token among processes arranged in a logical ring, ensuring fair and orderly access to the critical section.

### ➤ Election Algorithms

#### May\_Jun\_2025

Q2) b) Explain the goal of an Election algorithm. Illustrate the bully algorithm using appropriate diagrams.

#### May\_Jun\_2023

Q2) b) Why do we need an election algorithm? Using appropriate diagrams describe the bully algorithm step-by-step.

Ans. Goal of an Election Algorithm

In distributed systems, multiple processes work together without a centralized controller. Often, one process is selected as a **coordinator (leader)** to perform special tasks such as:

- Resource management
- Synchronization
- Scheduling
- Mutual exclusion coordination

An **election algorithm** is used to select a new coordinator when the current coordinator fails.

## Why Do We Need an Election Algorithm?

A coordinator process may fail because of:

- System crash
- Network failure
- Hardware problems

Without a coordinator:

- System coordination stops
- Resource management fails
- Distributed operations become inconsistent

Therefore, processes must elect a new coordinator automatically.

### Objectives of Election Algorithm

#### 1. Select a Unique Coordinator

Only one process should become leader.

#### 2. Detect Coordinator Failure

Processes should identify crashed coordinators.

#### 3. Ensure System Continuity

Distributed system should continue functioning after failures.

#### 4. Avoid Conflicts

Multiple coordinators should not exist simultaneously.

### Bully Algorithm

The **Bully Algorithm** is a distributed election algorithm proposed by Garcia-Molina.

It selects the process with the **highest process ID** as the coordinator.

Higher-ID processes can “bully” lower-ID processes during election.

## Assumptions of Bully Algorithm

- Each process has a unique ID.
- Higher ID  $\Rightarrow$  Higher priority.
- Every process knows IDs of all other processes.
- Processes can detect coordinator failure.
- Communication is reliable.

## Working Principle

When a process detects coordinator failure:

1. It starts an election.
2. It sends election messages to all higher-ID processes.
3. If no higher process replies, it becomes coordinator.
4. Otherwise, higher processes continue the election.

## Example System

Suppose there are five processes:

P1    P2    P3    P4    P5

where:

P5 has highest priority

Initially:

Coordinator = P5

Assume P5 crashes.

## Step-by-Step Working of Bully Algorithm

### Step 1: Failure Detection

Suppose P2 detects that coordinator P5 has failed.

P2 detects P5 failure

### Step 2: Election Message

P2 sends **ELECTION** messages to all higher-ID processes:

P2 → P3, P4, P5

### Diagram

P2 ----ELECTION----> P3

P2 ----ELECTION----> P4

P2 ----ELECTION----> P5

### Step 3: Responses

Higher active processes reply with:

OK message

Suppose:

- P3 replies
- P4 replies
- P5 is crashed

P3 → OK

P4 → OK

#### Step 4: Higher Process Starts Election

Now P4 starts its own election because it has higher priority.

P4 sends election messages to:

P5

#### Diagram

P4 ----ELECTION----> P5

No reply is received because P5 failed.

#### Step 5: New Coordinator Selection

Since no higher process responds, P4 becomes coordinator.

Coordinator = P4

#### Step 6: Coordinator Announcement

P4 broadcasts:

COORDINATOR message

to all lower processes.

#### Final Diagram

P4 becomes Coordinator

```
+-----+
|  COORDINATOR msg  |
+-----+
```

P4 → P1

P4 → P2

P4 → P3

### Complete Bully Algorithm Flow

1. Detect coordinator failure
2. Send ELECTION to higher-ID processes
3. Receive OK responses
4. Higher process starts new election
5. Highest alive process wins
6. Winner sends COORDINATOR message

### Advantages of Bully Algorithm

#### 1. Simple to Implement

Easy algorithm for distributed systems.

#### 2. Fast Coordinator Selection

Highest-ID active process quickly becomes leader.

#### 3. Ensures Unique Coordinator

Only one highest active process wins.

### Disadvantages

#### 1. High Message Overhead

Many election messages are exchanged.

## 2. Poor Scalability

Performance decreases with many processes.

## 3. Assumes Reliable Failure Detection

Incorrect failure detection may cause unnecessary elections.

## 4. Recovery Problem

Recovered higher-ID process may restart elections repeatedly.

## Message Complexity

Worst case:

$$O(n^2)$$

messages may be exchanged.

## Applications

- Distributed operating systems
- Cluster management
- Distributed databases
- Resource coordination systems

## Conclusion

The goal of an election algorithm is to select a coordinator in distributed systems when the current leader fails. The Bully Algorithm elects the active process with the highest process ID through election and coordinator messages, ensuring proper coordination and continuity of distributed operations.

### ➤ Gossip Based Coordination

## Gossip-Based Coordination in IPC

**Gossip-based coordination** is a decentralized communication technique used in distributed systems where processes exchange information periodically with a few randomly selected processes, similar to how gossip spreads among people.

It is commonly used for:

- Information dissemination
- Failure detection
- Synchronization
- State sharing
- Distributed coordination

In IPC and distributed systems, gossip protocols help processes coordinate without relying on a centralized controller.

## Basic Idea of Gossip Protocol

Each process:

1. Maintains some local information
2. Periodically selects another process randomly
3. Exchanges information with that process
4. Information gradually spreads throughout the network

## Working Principle

Process A → Process B

Process B → Process C

Process C → Process D

Information propagates step-by-step until all nodes receive it.

## Characteristics of Gossip-Based Coordination

- Distributed and decentralized
- Probabilistic communication
- Scalable
- Fault tolerant
- Eventually consistent

## Types of Gossip Protocols

## 1. Anti-Entropy Protocol

Processes periodically compare and update their data to remove inconsistencies.

## Purpose

- Database synchronization
- Replica consistency

## 2. Rumor Mongering Protocol

Information is spread like rumors until all nodes become informed.

## Purpose

- Event notification
- Update dissemination

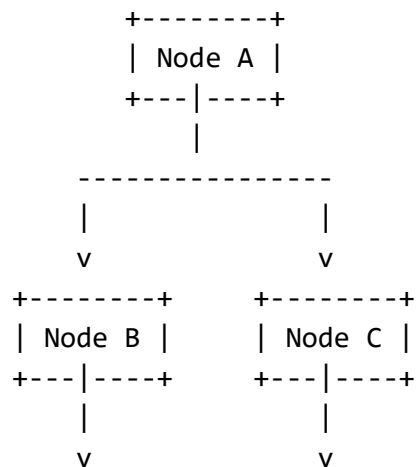
### 3. Aggregation Gossip

Used to compute global system statistics.

## Examples

- Average load
- Network size
- Resource usage

## Gossip Coordination Architecture



```
+-----+      +-----+
| Node D |      | Node E |
+-----+      +-----+
```

Each node exchanges information with neighboring/random nodes.

### Working Example

Suppose:

- Node A detects a system update.
- A sends update to B and C.
- B forwards to D.
- C forwards to E.

Eventually all nodes receive the update.

### Gossip-Based Coordination Algorithm

#### Step 1: Select Peer

Each node randomly selects another node.

#### Step 2: Exchange Information

Nodes exchange:

- State information
- Membership lists
- Failure information

#### Step 3: Update Local State

Processes update their own data.

#### Step 4: Repeat Periodically

The process repeats continuously.

## Advantages of Gossip-Based Coordination

### 1. Scalability

Works efficiently in very large distributed systems.

### 2. Fault Tolerance

No central coordinator means no single point of failure.

### 3. Decentralization

All nodes participate equally.

### 4. Simple Implementation

Easy protocol design.

### 5. Robust Communication

Information eventually reaches all active nodes.

## Disadvantages

### 1. Probabilistic Delivery

No guarantee of immediate delivery.

### 2. Redundant Messages

Same information may circulate multiple times.

### 3. Temporary Inconsistency

All nodes may not have identical information instantly.

### 4. Increased Network Traffic

Frequent gossip exchanges create overhead.

## Applications of Gossip Protocols

- Distributed databases
- Blockchain systems
- Cloud computing
- Failure detection
- Peer-to-peer networks
- Cluster management

## Example Systems Using Gossip

- Apache Cassandra
- Amazon Dynamo
- Kubernetes cluster coordination
- Distributed caching systems

## Gossip vs Centralized Coordination

| Feature                 | Gossip-Based  | Centralized    |
|-------------------------|---------------|----------------|
| Control                 | Decentralized | Central server |
| Scalability             | High          | Limited        |
| Fault Tolerance         | High          | Lower          |
| Complexity              | Moderate      | Lower          |
| Single Point of Failure | No            | Yes            |

## Conclusion

Gossip-based coordination is a decentralized IPC and distributed-system coordination mechanism where processes exchange information periodically with other nodes. It provides scalability, robustness, fault tolerance, and efficient coordination in large distributed environments without relying on a central coordinator.

## ◆ Case Study

## ➤ IBM WebSphere Message Queuing

### Case Study: IBM WebSphere Message Queuing (WMQ)

#### Introduction

**IBM WebSphere Message Queuing (WMQ)**, now known as **IBM MQ**, is a middleware messaging system developed by IBM for reliable and secure communication between distributed applications.

It enables applications running on different platforms to communicate asynchronously using message queues.

IBM MQ is widely used in:

- Banking systems
- E-commerce
- Airline reservation systems
- Distributed enterprise applications
- Cloud and microservices environments

#### Need for IBM MQ

In distributed systems:

- Applications may run on different machines
- Systems may use different operating systems
- Direct communication may fail during network issues

IBM MQ solves these problems by providing:

- Reliable message delivery
- Asynchronous communication
- Platform independence
- Guaranteed message transfer

#### Basic Concept

Applications communicate indirectly through queues.

Application A → Queue → Application B

The sender and receiver do not need to run simultaneously.

## Architecture of IBM MQ

### Main Components

#### 1. Queue Manager

The core component responsible for managing queues and message routing.

#### 2. Message Queue

Temporary storage area where messages are stored until processed.

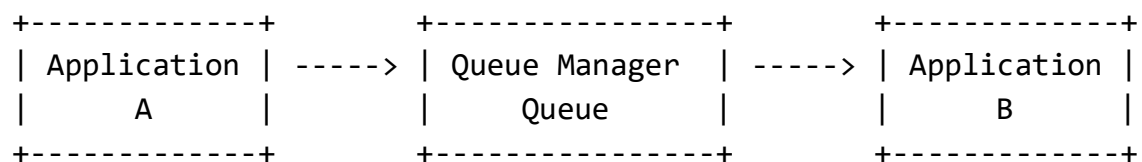
#### 3. Channels

Communication links between queue managers.

#### 4. Applications

Programs that send and receive messages.

### IBM MQ Architecture Diagram



Sender Side

Receiver Side

## Working of IBM MQ

### Step 1: Message Creation

Sender application creates a message.

### Step 2: Put Message into Queue

Message is stored in a queue using MQ API.

`MQPUT()`

### Step 3: Queue Manager Handles Delivery

Queue manager stores and routes the message.

### Step 4: Receiver Retrieves Message

Receiver application gets the message.

`MQGET()`

## Types of Queues in IBM MQ

### 1. Local Queue

Stores messages locally.

### 2. Remote Queue

Represents queues located on another queue manager.

### 3. Transmission Queue

Temporarily stores messages during transfer.

#### 4. Dead Letter Queue

Stores undeliverable messages.

### Communication Modes

#### 1. Point-to-Point Messaging

Sender → Queue → Single Receiver

#### 2. Publish/Subscribe Messaging

Publisher → Topic → Multiple Subscribers

### Features of IBM MQ

#### 1. Asynchronous Communication

Applications communicate independently.

#### 2. Reliable Message Delivery

Messages are not lost even during failures.

#### 3. Platform Independence

Supports:

- Linux
- Windows
- UNIX
- Mainframes

#### 4. Security

Provides:

- Authentication
- Encryption
- Access control

## 5. Transaction Support

Ensures data consistency during failures.

## 6. Scalability

Supports large enterprise systems.

## Advantages of IBM MQ

### 1. Guaranteed Delivery

Messages are safely delivered.

### 2. Loose Coupling

Applications remain independent.

### 3. Fault Tolerance

Messages survive crashes and network failures.

### 4. Improved Reliability

Supports enterprise-grade communication.

### 5. Simplified Integration

Connects heterogeneous systems easily.

## Disadvantages

### 1. High Cost

Enterprise licensing can be expensive.

### 2. Complex Configuration

Requires skilled administration.

### 3. Resource Consumption

Consumes memory and storage for queues.

## Applications of IBM MQ

- Banking transactions
- ATM networks
- Airline reservation systems
- Stock trading platforms
- E-commerce systems
- Cloud integration
- Enterprise service buses (ESB)

## Example Use Case

### Banking System

ATM Machine → IBM MQ Queue → Bank Server

Even if the bank server is temporarily unavailable, messages remain safely stored in queues.

## IBM MQ APIs

IBM MQ provides APIs for:

- C
- C++
- Java
- Python
- JMS

Common operations:

MQPUT() → Send message

MQGET() → Receive message

### IBM MQ vs Traditional Communication

| Feature         | IBM MQ | Direct Communication |
|-----------------|--------|----------------------|
| Coupling        | Loose  | Tight                |
| Reliability     | High   | Lower                |
| Asynchronous    | Yes    | Limited              |
| Fault Tolerance | High   | Lower                |
| Scalability     | Better | Limited              |

### Conclusion

IBM WebSphere Message Queuing (IBM MQ) is a powerful middleware messaging system that enables reliable, asynchronous, and secure communication between distributed applications using message queues. It supports scalable enterprise integration, fault tolerance, and guaranteed message delivery, making it widely used in modern distributed systems and enterprise applications.